

第9章

关系数据库存储管理

本章讨论关系数据库的数据组织和存储管理。我们将重点介绍基于磁盘的数据库的组织与存储，有关内存数据库的数据组织见新技术篇的第17章。

本章导读

存储管理是数据库管理系统(DBMS)的重要职责之一。它与查询优化以及数据库物理设计都密切相关。

本章介绍关系数据库的数据组织和索引结构。在9.1节数据组织中主要讨论数据库的逻辑与物理组织方式，重点介绍记录的表示、记录如何在块中组织存储以及如何组织关系表的存放；在9.2节索引结构中重点介绍5类索引，包括顺序表索引、辅助索引、B+树索引、哈希索引和位图索引。

本章的难点是理解数据库逻辑组织与物理组织之间的对应关系，了解关系表的几种存放方式，深入理解索引的作用，掌握B+树索引和哈希索引的组织方式和查找方式。

在第一篇基础篇中我们已经知道，关系数据库的一个重要特点是存储结构和存取路径对用户隐蔽，用户访问数据库时，只需要用SQL语句表达自己需要哪些信息，比如“查询数据库系统概论课程的选课人数”，而不需要像过程化语言那样具体指定怎么去查找。将用户的查询请求转化成查询执行计划，是数据库查询优化的一个重要工作，而了解关系数据库的数据组织与存储管理，能帮助我们更好地理解关系数据库查询优化的原理。同时，了解数据组织与存储管理，也能帮助我们在应用开发中更好地进行数据库物理设计。

9.1 数据组织

数据库是大量数据的有结构的综合性集合。如何将这样一个庞大的数据集合以最优的形式组织起来存放在外存上，是一个非常重要的问题。所谓“优”应包括两方面：一是存储效率高，节省存储空间；二是存取效率高，访问速度快，代价小。本节主要介绍数据库的逻辑组织方式和物理存储方式。

9.1.1 数据库的逻辑组织方式与物理组织方式

数据库数据存放的基础是文件，对数据库的任何操作最终都要转化为对文件的操作。所以在数据库的物理组织中，基本问题就是如何利用操作系统提供的基本文件组织来设计数据库数据的存放方法，这实际上也就对应了数据存储管理的两种方式。

第一种方式是每一个数据库对象（例如每一个基本表、索引等）都对应一个操作系统的文件，这种数据组织方式本质上是存储管理交由操作系统去完成。PostgreSQL、Kingbase ES 等采用此种存储管理方式。

第二种方式是整个数据库对应一个或若干个文件，由数据库管理系统进行存储管理，这种存储方式也称作段页式存储方式。Oracle、SQL Server 等采用这种存储管理方式。

在段页式存储管理中，数据库管理系统首先向操作系统申请一个大文件，当数据量不断增加，文件空间不够时，数据库管理系统会向操作系统申请追加新的文件。为了方便管理，数据库管理系统通常会对数据库空间进行逻辑划分，以增加灵活性。不同数据库管理系统从逻辑上划分数据库空间的方法并不完全一样，一种常见的划分方式是将数据库组织成“表空间-段-分区-数据块”的形式，如图 9.1 所示。

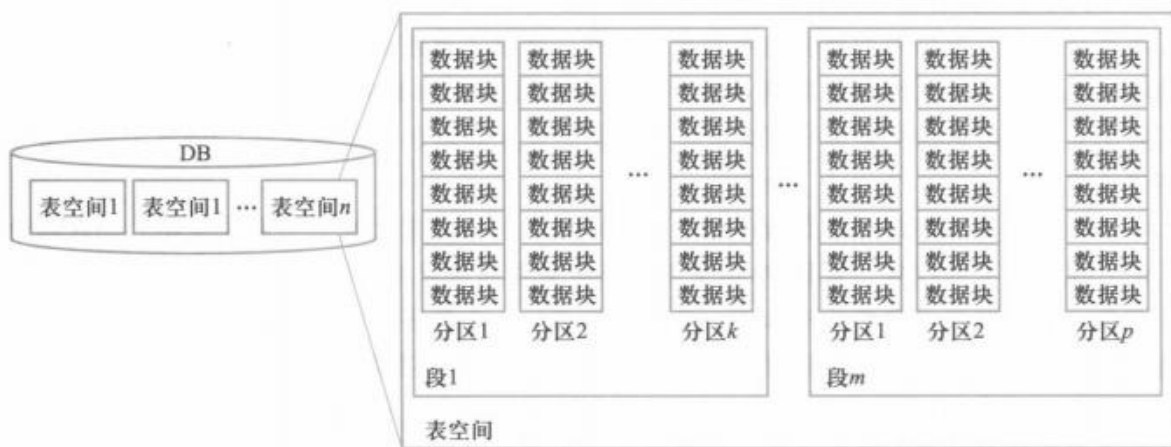


图 9.1 一种数据库的逻辑组织方式

在图 9.1 中，一个表空间对应磁盘上一个或多个物理文件，但一个物理文件只能属于一个表空间。一个数据库可以有多个表空间，从逻辑上组织数据库中的数据存储，如系统表空间、联机表空间、临时表空间等。一个表空间逻辑上由多个段组成，每个段可以逻辑上组织不同类型的数据，如数据段、索引段、临时段等。一个段逻辑上又由多个分区组成，每个分区由一组连续的数据块组成。块是数据库的磁盘存取单元，其大小为操作系统块的整倍数。

从逻辑上划分数据库空间是为了方便数据的管理。从物理上看，数据库中的数据最终是以文件（关系表）的形式存储在磁盘上。每个文件物理上分成定长的存储单元，即操作系统的物理块。物理块是存储分配和 I/O 处理的基本单位。一个物理块可以存放表中的多个记录（元

组), 一个表通常会占用多个块。因此数据库的物理组织形式是“文件-块-记录”。图 9.2 是逻辑组织方式与物理组织方式之间的对应关系。接下来, 9.1.2 小节将具体介绍如何组织单条记录, 9.1.3 小节将介绍如何在一个块中存放一组记录, 9.1.4 小节将介绍如何组织一个关系表在块中的存储。



图 9.2 数据库逻辑组织方式与物理组织方式的对应关系

9.1.2 记录表示

关系表的记录可以以定长记录和变长记录两种形式来存储。

1. 定长记录存储

以定长记录方式存储数据, 指的是关系表中每条记录占据相同大小的空间。即使表中有变长字段, 也可以以定长记录方式存储, 这时会为变长字段预留最大长度的空间。

例如, 对于第 3 章例 3.5 创建的 Student(Sno, Sname, Ssex, Sbirthdate, Smajor) 表, 采用定长记录存储方式, 其一条记录的存储形式如图 9.3 所示。这里给 VARCHAR 数据类型的 Sname、Smajor 属性分配了最大长度的空间。

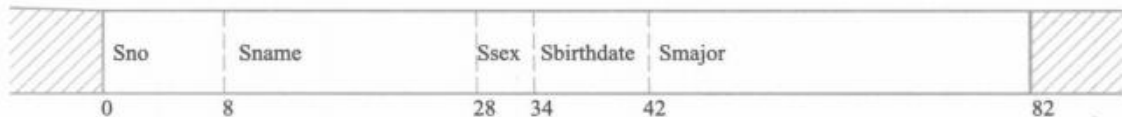


图 9.3 定长记录存储

有些硬件系统对内存中数据的起始地址有要求, 比如 4 的倍数或 8 的倍数。如果数据在磁盘上如图 9.3 这样紧密存储, 在读入内存时就需要做地址转换。为了简化地址转换, 并方便系统在不同平台上移植, 可以在外存中也保证各字段的起始地址是 4 或 8 的倍数, 如图 9.4 所示。在图 9.4 中, 每个字段都是从 8 的倍数的地址起始, 虽然 Sname 的定义是 20 B, 但实际分配给它 24 B。同样, 本来定义了 6 B 的 Ssex 属性, 这里分配了 8 B。

采用定长记录存储方式的好处是, 能快速定位到记录及其属性的物理位置, 增、删、改比较方便和快捷; 但这种方式显然会浪费一些存储空间。

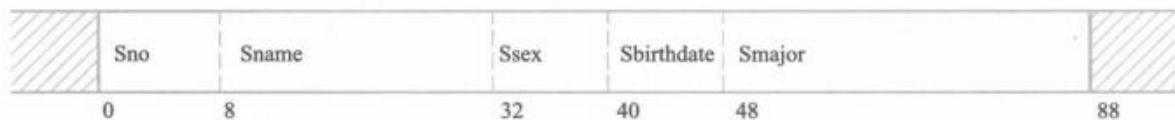


图 9.4 定长记录存储(字段起始地址为 8 的倍数)

2. 变长记录存储

Student 表的 Sname 属性和 Smajor 属性都是变长字符串, 所以 Student 表也可以以变长记录形式存储, 以节省存储空间。在组织变长记录时, 应保证能够快速访问一条记录, 并能快速地访问其中的所有属性, 包括定长和变长属性。通常有三种存储变长记录的方式。

第一种方式是在每条记录的头部记录该条记录的长度, 并在这条记录的每个变长字段前记录该字段的长度, 如图 9.5 所示。在这个例子中, 假设中文字符采用 GBK 编码, 每个汉字占 2 B。



图 9.5 变长记录存储(加长度标记)

第二种方式是先存储定长字段, 后存储变长字段, 第一个变长字段紧随定长字段之后, 从第二个变长字段开始, 在记录首部用指针(偏移量)指向变长字段, 如图 9.6 所示。



图 9.6 变长记录存储(先存储定长字段后存储变长字段)

第三种方式是将定长字段与变长字段分开存储在不同的块中, 图 9.7 示例了这种存储方式, 但实际上这种方式更多应用于 BLOB^① 等类型数据的存储。这种存储方式的好处是, 如果经常访问的是定长字段, 则可以减少数据存取时的 I/O 数量。

① BLOB 即 binary large object, 二进制大对象

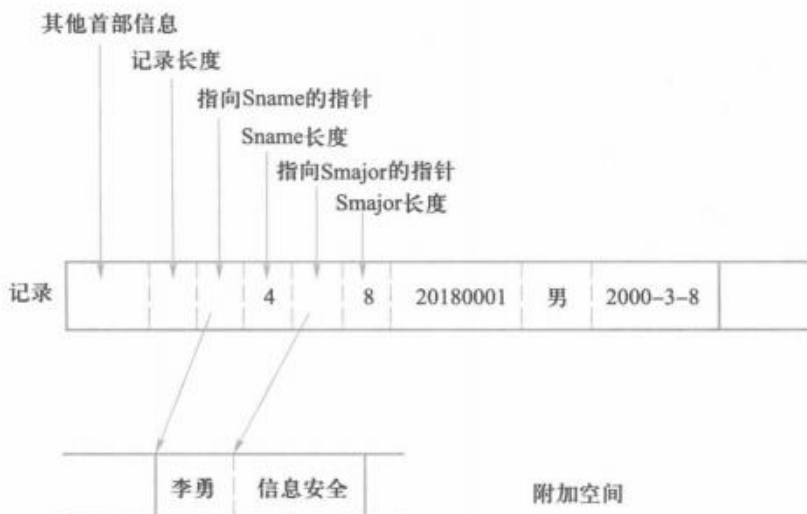


图 9.7 变长记录存储(定长字段与变长字段分开存储)

9.1.3 块的组织

定长记录和变长记录如何在块中组织存储呢?

1. 定长记录存储的块组织

定长记录的存储方式在实现上相对比较简单, 可以把一个表中的记录依次存放在块中。

图 9.8 是 Student 表采用定长记录存储方式时存放在一个物理块中的示意图, 每个块的首部可以有块头信息, 记录该块的 ID、最后一次修改和访问该块的时间戳、每条记录在块内的偏移量、空闲空间的头指针等。



图 9.8 定长记录存储的块组织

以定长方式存储记录, 需修改记录值时可直接在原位置修改, 不会出现原位置存放不下需要迁移记录的情形。删除记录时空间回收也很简单, 只需要将空闲空间加入空闲空间链表中即

可,不需要移动空闲空间,因为后面插入的新记录,其大小与被删除的记录是一样的,可以直接占用被删除记录的空间。图 9.9 是在图 9.8 所示的 Student 表中删除 20180003 和 20180006 两条记录后的情形。



图 9.9 定长记录存储的块组织(删除记录后)

2. 变长记录存储的块组织

对于变长记录的存储,块的组织就要稍复杂一点了,为了能够在块中快速定位到一条变长记录,通常需要在块头部分存放各条记录的指针(块内偏移量),空闲空间集中存储在块的中间部分,同时在块头有一个空闲空间尾指针(块内偏移量),以便在插入记录时快速找到空闲空间,如图 9.10 所示。



图 9.10 变长记录存储的块组织

在块中组织变长记录的存储时,记录从块的尾部开始连续存放。插入新记录时,从空闲空间尾部为其分配空间,在偏移量表中记录该记录的起始位置,并调整空闲空间尾指针。

删除一条记录时,会在偏移量表中为该记录指针置删除标记,释放其所占用的空间,并移动物理位置在其前面的记录,以保证空闲空间连续。相应地,被移动的记录在偏移量表中的指针以及空闲空间尾指针也需做相应修改。由于物理块通常都比较小,移动记录的开销并不高。

修改记录时,如果修改后记录在原位置存放不下,也会带来记录的迁移。

9.1.4 关系表的组织

了解了如何在一个块中存储一组记录之后,接下来的问题是如何组织关系表的存储。一个块只存储一个表的记录还是可以同时存储多个表的记录,这些记录按什么顺序存放,这些都是本节要讨论的问题。目前常见的关系表组织方式包括以下几种:堆存储、顺序存储、多表聚簇存储、B+ 树存储和哈希存储等。

1. 堆存储

表中的一条记录可以存储在该表的任何块中,没有顺序要求。在插入记录时,只要在该表的块中找到合适的空闲空间即可。如果没有足够的空闲空间,就为该表申请新的块。

2. 顺序存储

这种方式是一个表中的各条记录按照指定的属性或属性组的取值大小顺序存储。在一个块内,记录按照排序属性(组)的取值物理排列,同一个表的不同块之间则通过指针链接实现有序。这种组织方式可以高效地处理按排序属性(组)进行查询的请求。例如,如果 Student 表按专业(Smajor)升序存储,则可以快速完成如下的 SQL 语句:

```
SELECT Sno, Sname, Ssex  
FROM Student  
WHERE Smajor='计算机科学与技术';
```

但这种方式在插入或修改记录时,为保持记录顺序,有可能需要在块内或块间迁移已有记录,因而代价较高。

3. 多表聚簇存储

在默认情况下,一个块只用来存储同一个表中的记录,不同表的记录分别存储在不同的块中。但在有些情况下,不同表的记录也可以聚簇存储在同一组块中,其目的是减少连接操作的开销。

例如,两个具有主外码参照关系的表 Student 和 SC 可以按照学号(Sno)相等进行聚簇存储。图 9.11 是这种存储方式的一个示意图,在每个 Student 记录之后是该学生的选课记录。

如果用户想查询 20180001 学生的学号、姓名、选课门数,其 SQL 语句如下:

```
SELECT SC. Sno, Sname, COUNT(*)  
FROM Student, SC  
WHERE Student. Sno=SC. Sno AND SC. Sno='20180001';
```

那么只需要先找到 Student 表的 20180001 记录,其后跟着的就是 SC 表中该学生的所有选课记录,对其进行计数即可得到查询结果。明显地,对 Student 表和 SC 表按 Sno 值相等聚簇存储,相当于对 Student 表和 SC 表进行了预连接,它能够加速 Student 表和 SC 表的连接查询。

但是聚簇存储也是一把“双刃剑”,如果用户想查询数据科学与大数据技术专业的所有女生信息:



图 9.11 多表聚簇存储示意图

```
SELECT *
FROM Student
WHERE Smajor='数据科学与大数据技术' AND Ssex='女';
```

其效率就会低于 Student 表单独存放的情形，因为 Student 的元组会分散在更多的块里，需要更多次 I/O 操作。此外，这种数据组织方式在更新操作时会带来更频繁的数据迁移。例如，新学期开始时所有学生都新选了多门课程，为了保证每个学生的选课记录跟在其学生信息之后，将会频繁地迁移记录，以腾出空间插入新的选课记录。

4. B+树存储

传统的顺序存储方式在对表中记录进行增、删、改操作时，为维护记录顺序会付出较大的代价。B+树存储是以 B+树索引的形式确定记录存储在哪个数据块中，这样能在保持较高的记录访问效率的同时降低数据维护开销。B+树索引将在 9.2.3 小节介绍。与 B+树索引的区别是，B+树存储方式中叶节点的数据块中存放的不是索引项，而是数据记录。

5. 哈希存储

哈希存储方式是用哈希函数计算表中指定属性的哈希值，以此确定相应记录存储在哪个块中。哈希存储有两个关键要素，哈希函数和哈希表。哈希表由 B 个哈希桶(bucket)组成，每个桶对应一个或多个物理块，并用一个 $0 \sim B-1$ 之间的整数作为桶编号，桶中可以存储一条或多条记录。哈希函数以记录的哈希属性为输入，为其计算出一个 $0 \sim B-1$ 之间的整数，这个整数对应的是桶号。采用这种技术，可以将记录的存储位置与其指定属性的取值之间建立一个对应关

系,即按照记录的哈希属性值对应到一个存储位置,哈希属性值相同的记录会存储在相同的数据块中。在查找数据记录时,通过计算其哈希属性值可以快速定位到这条记录的位置。

9.2 索引结构

在很多情况下,用户的查询请求只涉及表中的少量记录,例如,查询计算机科学与技术专业的全体学生、查询李勇的出生日期等。当 Student 表很大时,如果读出该表的全部元组再逐条判断是否满足查询条件就会比较低效。

可以想象一下,当在一本书中查找某个特定的概念时,比较快捷的方法是通过该书的目录直接定位到想要查找的内容。如果关系表也能有一个类似于图书目录的附加结构,显然可以提高查询处理效率。这个附加结构就是索引。

索引的优势在于以下几个方面:

- ① 一个表的索引块数量通常比数据块数量少得多,因而搜索起来就会比较快。
- ② 索引通常采用一些易于检索的数据结构,可以使用高效的方法在索引中快速查找。
- ③ 对于经常访问的数据库表,如果其索引文件足够小则可以让其长久地驻留在内存缓冲区中,从而减少了 I/O 操作。

当然,索引也会带来额外的开销,包括存储空间的开销、建立索引的开销,以及当对基本表进行增、删、改操作时维护索引的开销。

数据库中常见的索引结构包括顺序表索引、辅助索引、B+树索引、哈希索引和位图索引等。下面依次介绍这几种索引结构。

9.2.1 顺序表索引

前面介绍了关系表的顺序存储方式。为了提高顺序表的查找效率,可以在顺序表的排序属性(组)上建立索引。这种索引结构是建在按索引属性值顺序存储的关系表上的,也称作主索引(primary index)或聚簇索引(clustering index)。它实际上就是第3章和第7章中提到的聚簇索引的一种形式。

注意:虽然称作主索引,但这种索引结构与主码(primary key)没有关系,索引属性可以是任意属性(组)。

顺序表上的主索引分为稠密索引和稀疏索引两类。索引结构中,每个索引项由索引属性的取值以及指向相应记录的指针两部分组成。与主索引相对应的是辅助索引(secondary index),将在9.2.2小节进行介绍。

1. 稠密索引

稠密索引(dense index)的索引块中存放每条记录的索引属性值以及指向相应记录的指针。以 Student 表 Sno 属性上的稠密索引为例,为方便举例,假设每个块只能存放2条 Student 表记录或6个 Sno 索引项,其索引如图9.12所示。其中,每条 Student 记录在索引中都对应了一个索引项。



图 9.12 稠密索引示例

如果用户想在 Student 表中查找学号为 20180012 的学生信息，其 SQL 语句如下：

```
Select *
From Student
WHERE Sno='20180012';
```

借助 Sno 属性上的稠密索引，经过 3 次 I/O 操作即可找到该记录：前两次 I/O 操作用于读入索引块，在其中查找 20180012 的索引项，第三次 I/O 操作根据索引项中的指针读取 20180012 元组。如果不借助索引而直接扫描基本表，则依次读入 Student 表的各个块，需要经过 6 次 I/O 操作才能找到 20180012 元组。

稠密索引是一个有序索引，当索引比较大时，可以用二分查找法在稠密索引中查找指定的索引项。

2. 稀疏索引

在稠密索引中，关系表的每条记录都对应了一个索引项。当关系表的记录数比较多时，索引会比较大，在索引中进行查找可能会花费较多的时间。稀疏索引 (sparse index) 就是为了解决这个问题而引入的。

在稀疏索引中，基本表的每个物理存储块只对应一个索引项，即稀疏索引的每个索引项存放每个物理块的第一条记录的索引属性值及指向该物理块的指针。仍以 Student 表为例，其稀疏索引如图 9.13 所示。

20180001		20180001	李勇	男	2000-3-8	信息安全
20180003		20180002	刘晨	女	1999-9-1	计算机科学与技术
20180005		20180003	王敏	女	2001-8-1	计算机科学与技术
20180007		20180004	张立	男	2000-1-8	计算机科学与技术
20180009		20180005	陈新奇	男	2000-11-1	信息管理与信息系统
20180011		20180006	赵明	男	2000-6-12	数据科学与大数据技术
20180013		20180007	王佳佳	女	2001-12-7	数据科学与大数据技术
20180015		20180008	张山	男	2001-5-1	数据科学与大数据技术
		20180009	李司	男	2000-7-8	计算机科学与技术
		20180010	王坤	女	2001-9-1	计算机科学与技术
		20180011	陈琪	女	2001-8-10	计算机科学与技术
		20180012	孙天天	女	2001-2-12	计算机科学与技术
		20180013	李昱	男	2000-10-1	信息管理与信息系统
		20180014	沈录	男	1999-12-2	信息管理与信息系统
		20180015	邓瑶	女	2001-2-22	信息安全

图 9.13 稀疏索引示例

如果用户要用稀疏索引查询 Student 表中学号为 20180012 的学生信息, 首先在稀疏索引中查找属性值小于或等于 20180012 的最大索引项(根据索引的大小, 用顺序查找法或二分查找法), 即 20180011 的索引项, 然后根据它的指针找到相应的存储块, 在该块中顺序搜索, 查找 Sno 属性值为 20180012 的记录。完成这个查询共需要 2 次 I/O 操作。如果在该块没有找到 Sno 为 20180012 的记录, 则 Student 表必定不存在这条记录, 因为下一个存储块的 Sno 值肯定大于 20180012。

稀疏索引除了尺寸小之外, 还有另外一个优势, 即当对基本表进行增、删、改时, 只要增加和删除的记录不是一个存储块的第一条记录, 修改的属性不是存储块第一条记录的索引属性, 则稀疏索引就不需要维护。

3. 多级索引

当关系表的数据量非常大时, 稀疏索引可能仍然比较大, 读取稀疏索引和在索引中查找的效率还不够高, 多级索引(multilevel index)就是为了解决这个问题而引入的。

多级索引中, 第一级索引是我们前面介绍的稠密索引或稀疏索引, 当这级索引较大时, 可以在其上再建第二级索引。如果第二级索引仍然较大, 可以在第二层索引上建第三级索引。以此类推, 直到索引尺寸合适为止。二级索引或更高级的索引必须是稀疏索引。图 9.14 为在 Student 表上建立的二级索引。

利用多级索引进行查找时, 从高级索引入手, 逐层向下, 直到定位到记录所在的物理块。例如, 在图 9.14 中查找学号为 20180014 的学生信息, 首先在第二级索引中按前面讲的稀疏索引的查找方法, 定位到 20180014 记录在一级索引中的索引块位置, 然后在一级索引块中查找

20180014 的索引项, 得到其记录指针。



图 9.14 多级索引示例

9.2.2 辅助索引

辅助索引 (secondary index) 是建立在表的非排序属性上的索引。一个表最多只能建立一个主索引, 但可以在不同的属性上建立多个辅助索引。由于辅助索引是建在无序属性上的, 所以它必须是稠密索引。图 9.15 是在 Student 表的 Smajor 属性上建立的辅助索引。

从图 9.15 可以看到, 建有辅助索引的属性往往会取重复值。如果能去掉重复取值的索引项则可以减小索引的大小, 减少在索引中查找的开销。但是不同索引属性值重复的次数往往不相同, 也很难预先估计重复次数, 因此不能简单地通过在每个索引项中预留多个指针来解决这个问题。一种常用的解决方案是用引入一个中间数据结构“指针桶”, 索引项中的指针指向指针桶中的相应位置, 指针桶中的指针指向相应元组, 如图 9.16 所示。这里假设一个物理块能存放 16 个桶指针, 所以在本例中指针桶占用一个物理块。

当在关系表上建立了多个辅助索引时, 可以利用辅助索引的指针桶来回答涉及多个属性条件的查询。例如, 对于如下查询:

```
SELECT *
FROM Student
WHERE Smajor='计算机科学与技术'
AND Sbirthdate='2002-1-1';
```

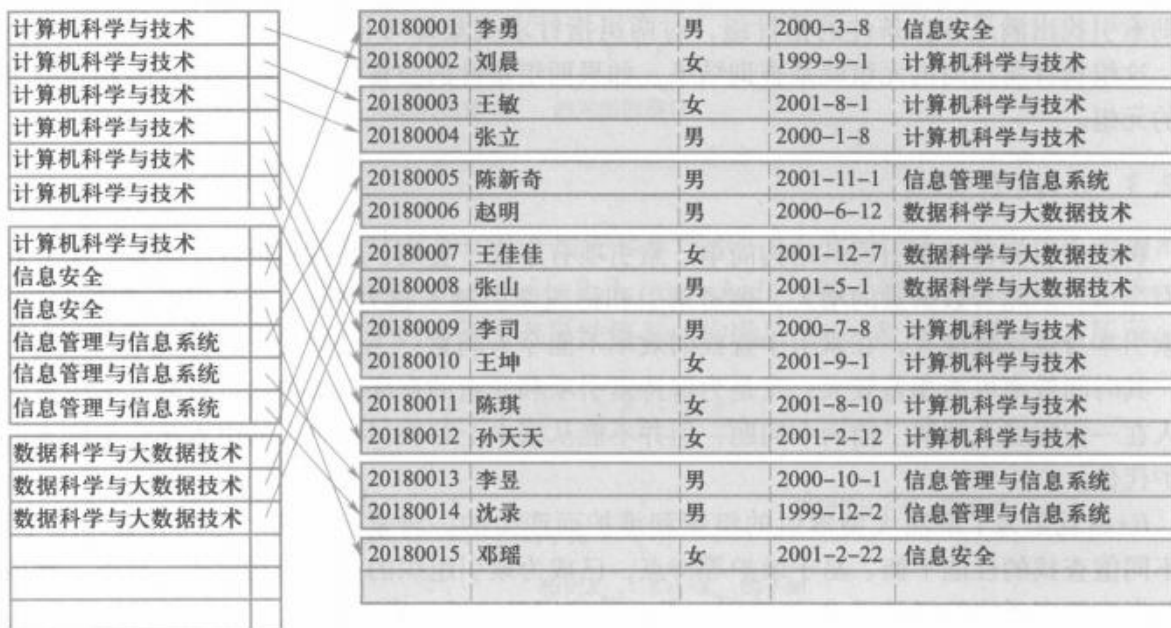


图 9.15 辅助索引示例

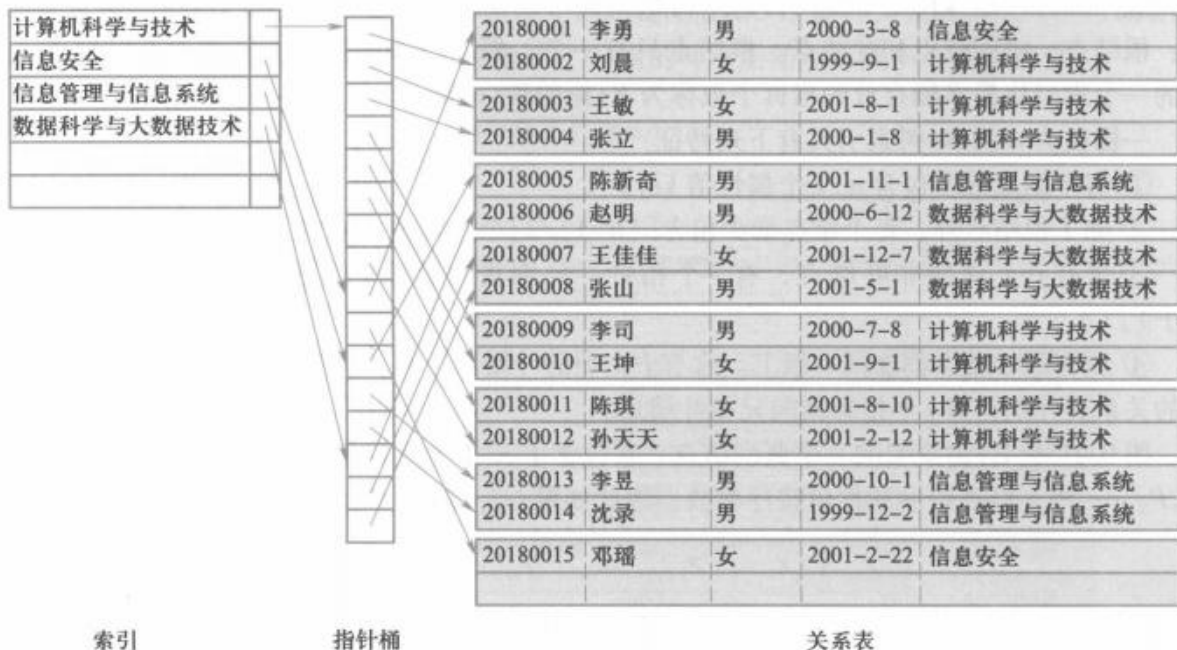


图 9.16 辅助索引示例(去掉重复索引项)

如果 Student 表在 Smajor 属性上和 Sbirthdate 属性上都建有辅助索引,则可以分别利用两个辅助索引找出满足各自条件的指针组,对两组指针求交集即可得到同时满足两个条件的指针组,这组指针所指向的元组就是查询结果。如果两组指针的交集为空,则说明没有满足查询条件的元组。

9.2.3 B+树索引

稠密索引和稀疏索引因其结构简单、索引项有序便于查询,在数据量较小时是一类有效的索引结构。但随着数据量的增大,稠密索引和稀疏索引越来越力不从心。一是大数据量的表,其索引本身也比较庞大,在索引中查找的效率不能令人满意;二是按同一属性的不同值进行查找,其时间效率可能相差较大;三是为保持索引项和元组的有序,维护代价较高。多级索引的引入在一定程度上缓解了前两个问题,但并不能从根本上解决,同时它的引入还会加剧索引的维护代价。

B+树就是为了解决大型索引的组织和维护而产生的一种索引结构。它具有查找效率高、按不同值查找的性能平衡、易于维护等特点,已成为索引组织的一种标准形式。目前主流的关系数据库管理系统都提供了 B+树索引,是一种使用最广泛的索引结构。

1. B+树索引的结构

B+树本质上是一个多级索引,但它不同于 9.2.1 小节中的多级顺序索引。B+树将索引块组织成一棵树,这棵树是平衡的,即从树根到树叶的所有路径都一样长。B+树的结点分为三类:根结点、中间结点和叶结点。根结点只有一个,根结点和中间结点统称为非叶结点。B+树的一个索引块最多能存放的指针个数称为 B+树的秩(order)。

一棵秩为 n 的 B+树索引具有下列特征:

- ① 每个结点最多包含 $n-1$ 个属性值 key。
- ② 除了根结点外,每个结点最少包含 $\lceil (n-1)/2 \rceil$ 个属性值 key(根结点最少含有一项)。
- ③ 含有 $j-1$ 项的非叶结点,有 j 个指针,分别指向其 j 个孩子(叶结点除外,它没有孩子)。
- ④ 所有的叶结点都在同一级上。含有 $j-1$ 项的叶结点,有 j 个指针,前 $j-1$ 个指针指向相应的关系表元组,第 j 个指针指向兄弟叶结点。

图 9.17 是 B+树索引的一个典型结点,它包含了 $n-1$ 个属性值 $K_1, K_2, \dots, K_{n-1}$ 和 n 个指针 $P_1, P_2, \dots, P_n$ 。结点中的属性值按序存放,因此如果 $i < j$,则 $K_i < K_j$ 。

P_1	K_1	P_2	$\dots$	P_{n-1}	K_{n-1}	P_n
-------	-------	-------	---------	-----------	-----------	-------

图 9.17 B+树索引的一个结点

对于叶结点,这里的指针 P_i 指向关系表中属性值为 K_i 的元组($i=1, 2, \dots, n-1$),指针 P_n 指向其兄弟叶结点。最后一个叶结点的 P_n 为空。如图 9.18 所示。

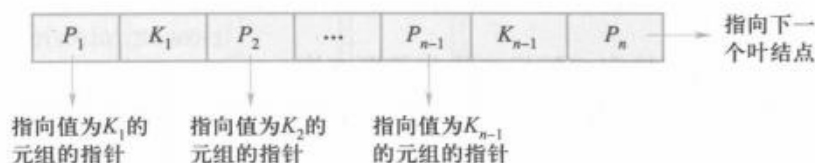


图 9.18 B+树索引的叶结点

对于非叶结点，这里的指针 P_i 指向其下层的子结点 ($i=1, 2, \dots, n$)。其中， P_1 指向的子树，其所有属性值 Key 均满足 $\text{Key} < K_1$ ； P_i ($i=2, 3, \dots, n-1$) 指向的子树，其所有属性值 Key 均满足 $K_{i-1} \leq \text{Key} < K_i$ ； P_n 指向的子树，其所有属性值 Key 均满足 $\text{Key} \geq K_{n-1}$ 。如图 9.19 所示。



图 9.19 B+树索引的非叶结点

图 9.20 是 Student 表 Sno 属性的 B+树索引，这里假设 n 为 5，即一个物理块最多能存放 4 个 Sno 属性值以及 5 个指针，最少要存放 2 个 Sno 属性值和 3 个指针，根结点除外。可以看到，借助各个叶结点中指向其兄弟叶结点的指针，B+树的叶子层结点形成一个顺序集合。

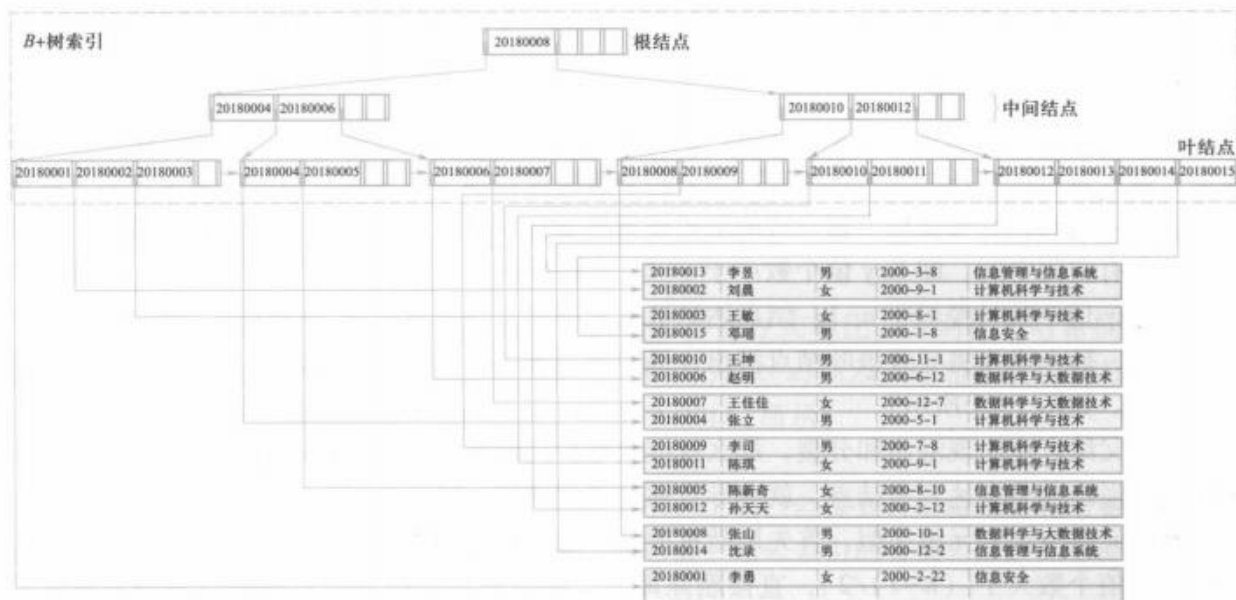


图 9.20 Student 表 Sno 属性上的 B+树索引

2. B+树索引的查询

借助 B+树索引可以高效地完成随机查找和范围查询。

随机查找是按照索引属性的取值进行查找。查询方法是：从根结点入手，根据要查询的属性值大小，沿着相应的父子结点指针逐层向下搜索，直到叶结点。如果在该叶结点中找到相匹配的属性值，则可用相应的元组指针取出查询结果；如果在该叶结点中没有找到匹配的属性值，则说明没有满足该条件的元组。

例如，在 Student 表中查询 $Sno = 20180011$ 的学生信息。首先在根结点中进行判断，由于 $20180011 > 20180008$ ，因此沿右子树继续搜索；在中间结点层，由于 $20180010 < 20180011 < 20180012$ ，因此沿 20180012 左侧指针继续向下搜索；在叶结点中找到 20180011 后，用其左侧的记录指针从数据库中取出 20180011 的元组。查询结束。

范围查询，例如查询 $Sno \geq 20180011$ 的学生信息，其方法是：首先用上面的随机查找方法找到 20180011 的元组，它是范围查找的入口点，以此开始顺序搜索，后面的所有属性值（包括本叶结点中的属性值和其后面的所有兄弟叶结点中的属性值）都满足查询条件，取出相应的元组指针即可。当前叶结点遍历完，沿指向下一叶结点的指针继续，直到最后一个叶结点。

对于条件为 between...and...的范围查询也类似，用随机查找方法分别找到范围条件的入口点和结束点，对入口点和结束点之间的属性值进行顺序搜索。

显然，B+树的查询路径是从根结点搜索到叶结点。由于 B+树是一棵平衡树，所有的叶结点都在同一层，因此无论查询条件中的属性值是什么，其查询效率都相似。例如，查询 $Sno = 20180011$ 的学生与查询 $Sno = 20180001$ 的学生，其 I/O 操作的次数是一样的。这是 B+树索引的优势之一。

3. B+树索引的维护

当对基本表执行插入或删除操作时，需要同时对该表上建立的所有 B+树索引进行相应的维护。维护 B+树索引的关键是要维持 B+树的平衡特性。

插入元组时，首先要用上面介绍的随机搜索算法找到相应索引项应插入哪个叶结点，如果该叶结点有空闲空间，即 Key 值个数小于 $n-1$ ，直接插入即可，这是最简单的情况。如果该叶结点的 Key 值个数已经等于 $n-1$ ，结点达到最大充满度，例如在图 9.20 中插入 $Sno = 20180016$ 的索引项，插入操作将导致该叶结点溢出，进而分裂成 2 个叶结点，分裂后，其父结点也要做插入操作。如图 9.21 所示。

如果父结点也出现溢出和分裂，那么就继续维护父结点的父结点，逐层向上。如果这种向上传递的溢出和分裂达到根结点，就会导致树的高度增加一层。

删除元组与插入元组类似。首先用随机搜索算法找到要删除的索引项。如果其所在的叶结点中 Key 值个数大于 $\lceil (n-1)/2 \rceil$ ，直接删除即可，删除后该叶结点仍然能满足最小充满度要求，这是最简单的情况。例如，在图 9.20 中删除 20180001 索引项就是这种情形。

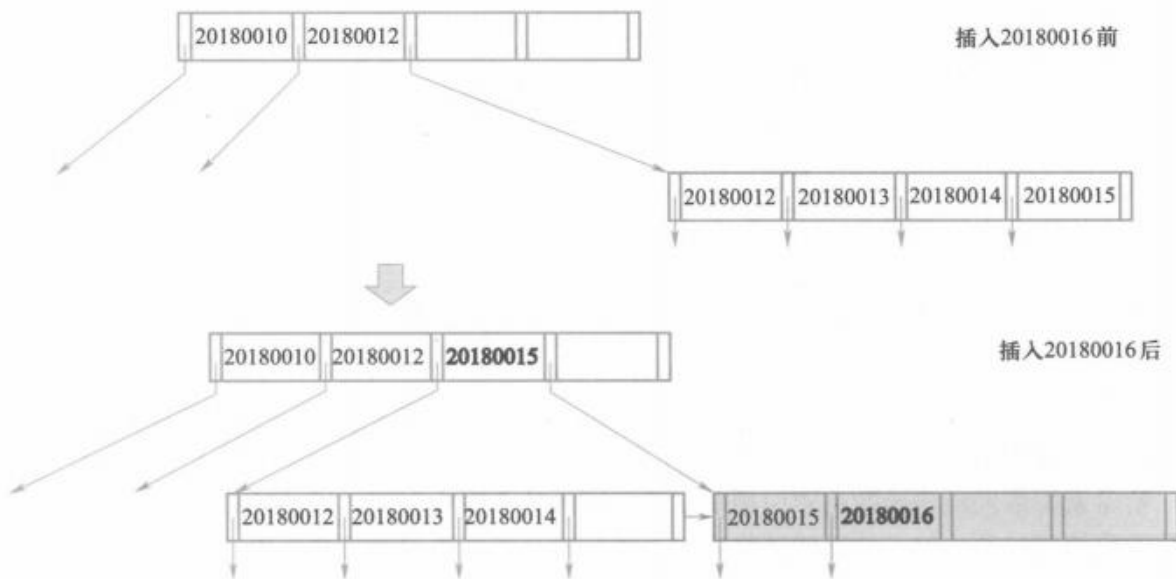


图 9.21 插入 Sno=20180016 前后的局部 B+树索引

如果删除后叶结点不能达到最小充满度，如在图 9.20 中删除 20180005 索引项，这时该叶结点就需要与其兄弟叶结点合并。叶结点合并后，其父结点也要执行相应的删除操作，这种删除操作导致父结点不能达到最小充满度，进而需要合并，最终导致树的高度降低一层。

有关 B+树索引的增、删、改操作，可以参见二维码内容。



微视频：B+树的
增、删、改操作

9.2.4 哈希索引

哈希索引是另一类能够实现快速查找的索引结构，该结构有两个关键要素，哈希函数和哈希表。哈希表由 B 个哈希桶组成。哈希函数用来将数据记录的索引属性值映射到哈希桶，即存放该记录索引的桶。每个桶存放一条或多条哈希值相同的索引项，每个索引项包含属性值和指向相应记录的指针。

哈希索引与 9.1.4 小节中介绍的关系表的哈希存储方式相比，区别在于哈希桶的存放内容。哈希存储方式中哈希桶存放的是数据记录，如图 9.22(a) 所示；哈希索引中哈希桶存放的是索引属性值及指向相应数据记录的指针，如图 9.22(b) 所示。

1. 静态哈希索引

静态哈希索引是最基本也是最简单的哈希索引。上面已经提到，哈希函数和哈希表是哈希索引的两个关键要素，接下来将从哈希函数、哈希表、哈希索引的查找与维护几方面介绍静态哈希索引。

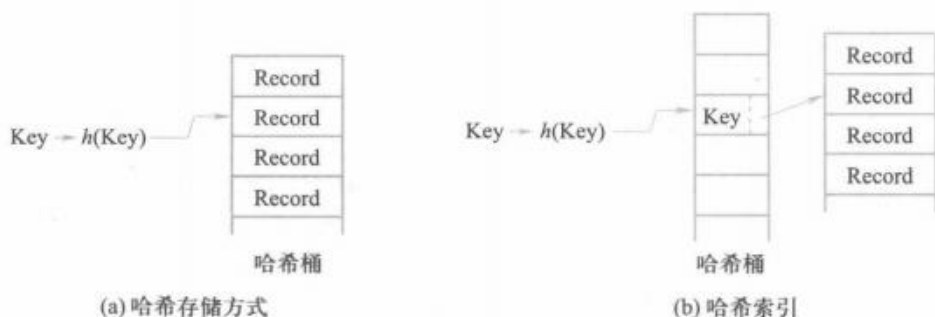


图 9.22 哈希存储方式与哈希索引

(1) 哈希函数

哈希索引借助哈希函数将索引项映射到不同的哈希桶中存储。设计哈希函数时要尽量保证函数的取值随机和均匀，从而能够将索引项均匀地分布到不同的哈希桶中，不会出现某个桶的索引项远远超过其他桶的情况。

例如，在 Student 表的 Sno 属性上建哈希索引，假设 Student 表有 5 000 条学生数据，Sno 索引有 100 个桶，哈希函数可以将学号转换为数值型，除以 100 取模，以此作为桶号；或者取学号最后两位，将其转换为数值型作为桶号。这两类哈希函数都可以比较均匀地将 Sno 索引项映射到 100 个桶中。

(2) 哈希表

哈希表由一组桶组成，一个桶对应一个或多个物理块。桶中存放被映射到该桶的哈希索引项，每条索引项包括索引属性值和指向相应记录的指针。图 9.23 是 Student 表 Sno 属性上的哈希索引，每个桶最多可以存放 6 个索引项。由于只有 15 条数据，这里设有 3 个哈希桶。哈希函数是取学号最后 2 位，将其转换为数值型，除以 3 并取模作为桶号。

哈希桶的空间是有限的，当某个桶的存储空间不足时将其称为桶溢出 (bucket overflow)。

出现桶溢出的原因主要有以下三类：

① 哈希桶的数量不足，不能存放所有的索引项。例如在图 9.23 中持续向 Student 表插入元组，当元组数量超过 18 条后，新的索引项就无处安放了。

② 属性取值不均衡，某些属性值过多。例如图 9.23 的 Student 表中计算机科学与技术专业的学生人数非常多，使得相应的桶无法容纳下所有的索引项。

③ 哈希函数设计不合理，无法将索引项均匀地映射到每个桶，导致某个桶的数据过多。

为了减少桶溢出的情况，数据库管理系统通常会预留出一定比例的空间。尽管如此，桶溢出还是不可能避免，必须有相应的处理措施。解决桶溢出最常用的方法是为该桶分配一个溢出桶，当溢出桶空间也充满时，再追加新的溢出桶。为了快速找到溢出桶，数据库管理系统会用溢出链将一个桶及其溢出桶链接在一起。如图 9.24 所示。



图 9.23 Student 表 Sno 属性上的哈希索引

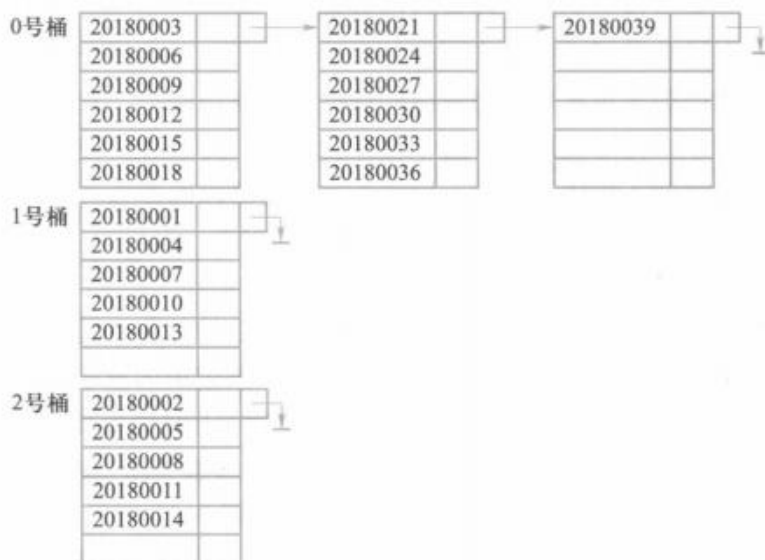


图 9.24 Student 表 Sno 属性上的哈希索引(带溢出链)

(3) 哈希索引的查找及维护

① 哈希索引的查找。哈希索引适用于等值查找。使用哈希索引进行查找时, 首先根据要查找的属性值计算出其哈希函数取值, 得到桶号, 然后去该桶中搜索相应的索引项。如果桶中没有找到, 但存在溢出桶, 还需要继续搜索溢出桶。

例如在图 9.24 中查找学号为 20180010 的学生, 首先用哈希函数计算出索引项 20180010 所在的桶号, 这里是 1 号桶, 然后去 1 号桶中查索引项 20180010, 如果找到, 则根据相应的记录指针取出学号为 20180010 的记录。如果 1 号桶中没有该索引项, 则说明不存在该学生。如果查找学号为 20180024 的学生, 则需要在 0 号桶中继续沿溢出链查找, 直到找到为止, 或者搜索完最后一个溢出块还没有找到, 则说明不存在该学生。

② 哈希索引的维护。向基本表中插入元组时, 需要同时向哈希索引中插入相应的索引项。方法是用哈希函数计算出索引项应插入的桶号, 如果该桶中有空间直接插入索引项, 如果该桶中空间已满则申请溢出桶并插入。

类似地, 删除基本表元组时, 需要找到其索引项所在桶并删除。如果有溢出块, 还需要判断一下是否有足够的空间可以合并溢出块。

2. 动态哈希索引

在静态哈希索引中, 桶的个数是事先确定好并且不再改变的。随着数据的不断增加, 桶溢出就成为静态哈希索引不可回避的问题, 而严重的桶溢出会极大地影响查找效率。前面提到的适当预留桶空间是一种解决方法。但实际情况是很难预先估算出一个表的数据量, 并预留合适的索引空间。预留空间过小不能解决桶溢出问题, 预留空间过大将会造成较大的空间浪费。

另一种方法是随关系表的增大而周期性地增加哈希桶的数目, 并重组索引。但索引重组是一个比较耗时的事情, 同时也会影响正在进行的查询。

动态哈希索引可以随着关系表的增大逐渐扩大桶的数目, 是更适合数据库应用特征的哈希索引结构。动态哈希索引包含可扩展哈希索引和线性哈希索引两类。它们都是将索引属性值散列到一个长的二进制位串, 然后使用其中若干位作为桶号, 并通过所使用的位数的增加来增加桶的个数。限于篇幅, 本章不再深入介绍, 感兴趣的读者可以阅读参考文献[8]~[13]。

9.2.5 位图索引

位图索引是长度为 n 的位向量集合, 其中 n 为索引属性的基数, 即它可能的取值个数。每一个位向量对应于索引属性中的一个可能取值。如果第 i 条记录的索引属性值为 v , 那么对应于值 v 的位向量在位置 i 上取值为 1, 其他的位向量在位置 i 上取值为 0。

例如, 对于 Student 表的性别属性 Ssex, 其基数为 2, 只有“男”和“女”两个可能的取值, 因而 Ssex 属性上的位图索引为长度为 2 的位向量集合, 如图 9.25 所示。第一条记录 20180001 是男生, 它所对应的位向量为“10”; 第二条记录 20180002 和第三条记录 20180003 均为女生, 两条元组所对应的位向量均为“01”; 依此类推。

当用户想查找 Student 表的所有女生时, 其 SQL 语句如下:

```
SELECT *  
FROM Student  
WHERE Ssex='女';
```

Ssno	Sname	Ssex	Sbirtdate	Smajor	男 女	
20180001	李勇	男	2000-3-8	信息安全	1	0
20180002	刘晨	女	1999-9-1	计算机科学与技术	0	1
20180003	王敏	女	2001-8-1	计算机科学与技术	0	1
20180004	张立	男	2000-1-8	计算机科学与技术	1	0
20180005	陈新奇	男	2001-11-1	信息管理与信息系统	1	0
20180006	赵明	男	2000-6-12	数据科学与大数据技术	1	0
20180007	王佳佳	女	2001-12-7	数据科学与大数据技术	0	1
20180008	张山	男	2001-5-1	数据科学与大数据技术	1	0
20180009	李司	男	2000-7-8	计算机科学与技术	1	0
20180010	王坤	女	2001-9-1	计算机科学与技术	0	1
20180011	陈琪	女	2001-8-10	计算机科学与技术	0	1
20180012	孙天天	女	2001-2-12	计算机科学与技术	0	1
20180013	李昱	男	2000-10-1	信息管理与信息系统	1	0
20180014	沈录	男	1999-12-2	信息管理与信息系统	1	0
20180015	邓瑶	女	2001-2-22	信息安全	0	1

图 9.25 Student 表 Ssex 属性上的位图索引

借助 Ssex 属性上的位图索引，第 2 位为 1 的所有记录就是满足查询条件的记录，取出对应的记录即可。

如果用户想统计男生和女生的人数，其 SQL 语句如下：

```
SELECT Ssex, Count(*)
FROM Student
GROUP By Ssex;
```

只需要分别统计 Ssex 属性上位图索引各位中 1 的个数即可，无须访问基本表。

如果在 Smajor 属性也建有一个位图索引，如图 9.26 所示，那么对于如下统计计算机科学与技术专业男生人数的查询：

```
SELECT count(*)
FROM Student
WHERE Ssex='男' AND Smajor='计算机科学与技术';
```

只需要将 Ssex 属性位图索引的第 1 位与 Smajor 属性位图索引的第 2 位进行与操作，得到结果向量

(0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0)

对结果向量中的 1 进行计数，即可获得满足条件的男生人数为 2。

位图索引还能有效处理多值查询。例如对如下查询：

```
SELECT *
FROM Student
WHERE Smajor in('信息管理与信息系统', '信息安全');
```


Ssno	Sname	Ssex	Sbirthdate	Smajor	数据 信息科学 计算机与 科学和大 数据科学 与大数据 安全技术 系统
20180001	李勇	男	2000-3-8	信息安全	1 0 0 0
20180002	刘晨	女	1999-9-1	计算机科学与技术	0 1 0 0
20180003	王敏	女	2001-8-1	计算机科学与技术	0 1 0 0
20180004	张立	男	2000-1-8	计算机科学与技术	0 1 0 0
20180005	陈新奇	男	2001-11-1	信息管理与信息系统	0 0 1 0
20180006	赵明	男	2000-6-12	数据科学与大数据技术	0 0 0 1
20180007	王佳佳	女	2001-12-7	数据科学与大数据技术	0 0 0 1
20180008	张山	男	2001-5-1	数据科学与大数据技术	0 0 0 1
20180009	李司	男	2000-7-8	计算机科学与技术	0 1 0 0
20180010	王坤	女	2001-9-1	计算机科学与技术	0 1 0 0
20180011	陈琪	女	2001-8-10	计算机科学与技术	0 1 0 0
20180012	孙天天	女	2001-2-12	计算机科学与技术	0 1 0 0
20180013	李昱	男	2000-10-1	信息管理与信息系统	0 0 1 0
20180014	沈录	男	1999-12-2	信息管理与信息系统	0 0 1 0
20180015	邓瑶	女	2001-2-22	信息安全	1 0 0 0

图 9.26 Student 表 Smajor 属性上的位图索引

只需要对 Smajor 属性位图索引的第 1 位和第 3 位进行或操作, 得到结果向量

(1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1)

结果向量中为 1 的记录就是满足条件的记录集合, 即第 1、5、13、14、15 条记录。

从上面这些例子可以看到, 借助位图索引, 可以使用位操作来快速回答用户查询或快速定位满足条件的元组集合, 从而减少了对基本表的全表扫描, 提高了查询效率。但是位图索引的大小与列的基数成正比, 基数大的列其位图索引会非常庞大, 因此它只适用于基数小的属性列。

编码位图索引(encoded bitmap index)对标准位图索引进行了改进, 通过对属性值编码以减少索引位向量的个数, 从而能够用于有较高基数的列。例如, Student 表 Smajor 属性的编码位图索引如图 9.27 所示。

显然, 如果索引属性列的基数为 K , 标准位图索引所需要的位向量个数为 K 个, 而编码位图索引所需要的位向量个数仅为 $\log_2 K$ 个。但是利用编码位图索引进行查询时, 需要访问所有位向量才能完成。例如, 在学生表中查找信息管理与信息系统专业和信息安全专业的所有学生, 需要扫描所有位向量, 从中查找编码为 10 或 00 的元组; 再如, 在学生表中查找信息管理与信息系统专业的所有学生, 也同样需要扫描所有位向量, 从中查找编码为 10 的元组。

Ssno	Sname	Ssex	Sbirthdate	Smajor		
20180001	李勇	男	2000-3-8	信息安全	0	0
20180002	刘晨	女	1999-9-1	计算机科学与技术	0	1
20180003	王敏	女	2001-8-1	计算机科学与技术	0	1
20180004	张立	男	2000-1-8	计算机科学与技术	0	1
20180005	陈新奇	男	2001-11-1	信息管理与信息系统	1	0
20180006	赵明	男	2000-6-12	数据科学与大数据技术	1	1
20180007	王佳佳	女	2001-12-7	数据科学与大数据技术	1	1
20180008	张山	男	2001-5-1	数据科学与大数据技术	1	1
20180009	李司	男	2000-7-8	计算机科学与技术	0	1
20180010	王坤	女	2001-9-1	计算机科学与技术	0	1
20180011	陈琪	女	2001-8-10	计算机科学与技术	0	1
20180012	孙天天	女	2001-2-12	计算机科学与技术	0	1
20180013	李昱	男	2000-10-1	信息管理与信息系统	1	0
20180014	沈录	男	1999-12-2	信息管理与信息系统	1	0
20180015	邓瑶	女	2001-2-22	信息安全	0	0

编码映射表

信息安全	00
计算机科学与技术	01
信息管理与信息系统	10
数据科学与大数据技术	11

图 9.27 Student 表 Smajor 属性上的编码位图索引

本章小结

在关系数据库中，虽然存储结构和存取路径对用户是隐蔽的，但了解关系数据库的存储管理技术，有助于更好地理解关系数据库查询优化的原理，也有助于在应用开发中更好地进行数据库物理设计。

本章主要讨论了关系数据库的数据组织方式和索引结构。介绍了数据库的逻辑与物理组织方式，讲解了如何物理地组织记录、块和关系表。系统讲解了顺序表索引、辅助索引、B+ 树索引、哈希索引和位图索引等索引结构。

习 题 9

1. 试分析每个数据库对象对应一个操作系统文件和整个数据库对应一个或若干个文件，这两种存储关系数据库的策略各有什么优缺点。

2. 假设 Course 表以定长记录方式存储。请描述 Course 表的记录存储，在以下情况下一条记录占多少字节？

- ① 字段可以在任何字节处开始。
- ② 字段必须在 4 的倍数的字节处开始。
- ③ 字段必须在 8 的倍数的字节处开始。

3. 试述关系表有哪些组织方式，并分析各自的优缺点。

4. 试述数据库索引机制的优点。

5. 试述稠密索引和稀疏索引的优缺点。

6. 分别描述利用图 9.12 的稠密索引、图 9.13 的稀疏索引、图 9.14 的多级索引、图 9.20 的 B^+ 树索引、图 9.23 的哈希索引是如何对 Student 表进行如下查询的。

- ① 查询学号为 20180013 的记录。
- ② 查询学号为 20180014 的记录。
- ③ 查询学号为 20180016 的记录。
- ④ 查询学号大于或等于 20180009 的记录。

7. 分别描述当用下列方式更新 Student 表中记录时, 图 9.12 的稠密索引、图 9.13 的稀疏索引、图 9.14 的多级索引、图 9.16 的辅助索引、图 9.20 的 B^+ 树索引、图 9.23 的哈希索引分别是如何进行维护的。

- ① 插入(20180016, 张婧宁, 女, 2002-1-2, 信息安全)记录。
- ② 删除学号为 20180004 的记录。
- ③ 删除学号为 20180005 的记录。
- ④ 将学号为 20180008 的记录修改为计算机科学与技术专业。

参考文献 9

[1] SILBERSCHATZ A, KORTH H F, SUDARSHAN S. Database system concepts[M]. 7th ed. McGraw-Hill Education, 2020.

[2] MOLINA H C, ULLMAN J D, WIDOM J. Database system implementation[M]. 2nd ed. Prentice Hall, 2008.

[3] WIEDERHOLD G. File organization for database design[M]. McGraw-Hill Education, 1987.

[4] PETERSON W W. Addressing for random-access storage[J]. IBM Journal of Research and Development, 1957, 1(2): 130-146.

[5] BAYER R. Symmetric binary B-trees: data structure and maintenance algorithms[J]. Acta Informatica, 1972, 1: 290-306.

[6] BAYER R, MCCREIGHT E. Organization and maintenance of large ordered indexes[J]. Acta Informatica, 1972, 1(3): 173-189.

Bayer 和 McCreight 在文献[6]中首先提出了 B 树存储结构, 现在 B 树已广泛应用于数据库的物理组织之中。

[7] COMER D. The ubiquitous B-tree[J]. Computing Surveys, 1979, 11(2): 121-137.

文献[7]对 B 树的历史和发展进行了系统阐述, 并介绍了各种 B 树的变种。

[8] FAGIN R, NIEVERGELT J, PIPPENGER N, et al. Extendible hashing—a fast access method for dynamic files[J]. ACM Transactions on Database Systems, 1979, 4(3): 315-344.

[9] LITWIN W. Linear hashing: a new tool for file and table addressing[C]. Proceedings of the 6th International Conference on Very Large Databases, 1980, 6: 212-223.

[10] RATHI H, LU H J, HEDRICK G E. Performance comparison of extendible hashing and linear hashing techniques[C]. Proceedings of the 1990 ACM SIGSMALL/PC Symposium on Small Systems, 1990, 17(2): 178-185.

[11] LITWIN W. Virtual hashing: a dynamically changing hashing[C]. Proceedings of the 4th International Conference on Very Large Data Bases, 1978, 4: 517-523.

- [12] BURKHARD W A. Hashing and trie algorithms for partial match retrieval[J]. ACM Transactions on Database Systems, 1976, 1(2): 175-187.
- [13] RAMAKRISHNA M V, LARSON P A. File organization using composite perfect hashing[J]. ACM Transactions on Database Systems, 1989, 14(2): 231-263.
- [14] NEIL P O, QUASS D. Improved query performance with variant indexes[C]. Proceedings of ACM SIGMOD International Conference on Management of Data, 1997: 38-49.
- [15] CHAN C Y, IOANNIDIS Y E. Bitmap index design and evaluation[C]. Proceedings of the 1998 ACM SIGMOD International Conference on Management of Data, 1998: 355-366.
- [16] WU M C, BUCHMANN A P. Encoded bitmap indexing for data warehouses[C]. Proceedings of the 14th International Conference on Data Engineering, 1998: 220-230.

本章知识点讲解微视频:



数据库的组织
方式



多表聚簇存放
方式



顺序表的索引



B+树索引



哈希索引

第 10 章 关系查询处理和查询优化

关系数据库的一个重要特点是存储结构和存取路径对用户隐蔽，用户访问数据库时，只需要用 SQL 语句表达自己需要哪些信息即可，而不需要像过程化语言那样具体指定怎么去做。将用户的查询请求转化成查询执行计划并高效地执行，是关系数据库管理系统的重要功能。本章将介绍关系数据库管理系统如何处理用户的查询请求，以及如何查询优化。

本章导读

关系数据库的查询处理和查询优化技术是数据库管理系统的重要功能，它极大地影响着关系数据库的性能。

本章首先介绍关系数据库管理系统的查询处理(query processing)步骤，然后深入讨论查询优化(query optimization)技术。查询优化一般可分为代数优化(也称为逻辑优化)和物理优化(也称为非代数优化)。代数优化是指关系代数表达式的优化，物理优化则是指通过存取路径和底层操作算法选择进行的优化。本章讲解实现查询操作的主要算法思想，目的是使读者初步了解关系数据库管理系统查询处理的基本步骤，以及查询优化的概念、基本方法和技术，为数据库应用开发中利用查询优化技术提高查询效率和系统性能打下基础。

学习完本章内容，要了解关系查询处理的常用算法，理解为什么关系数据库要进行查询优化，掌握查询优化的几种方法，其中重点和难点是深入理解基于代价估算的关系查询物理优化策略。

10.1 关系数据库管理系统的查询处理

查询处理是关系数据库管理系统执行查询语句的过程，其任务是把用户提交的查询语句转换为高效的查询执行计划并执行。

10.1.1 查询处理步骤

关系数据库管理系统的查询处理可以分为 4 个阶段：查询分析、查询检查、查询优化和查